Final Rundown: Performance Engineering for Modern LLM Inference

Contents

1 General Overview

Important

Interview pattern: Forward pass creates activations → loss measures error → backprop computes gradients → optimizer updates W, b (controlled by η and regularization).

Important

Driving scenario (preserved): A very large model (for example, ∼700B parameters) runs at about 3 tokens/s and latency is too high. You cannot solve this by only adding GPUs. Performance engineering means forcing the system through real hardware limits until it meets latency and throughput goals.

2 High-Probability Interview Q&A (Role Specific)

Question	Strong answer (concise)
How would you optimize LLM inference end-to-end?	Baseline first, split prefill/decode metrics, find top kernels with Nsight, optimize memory path (KV cache + fusion), then validate quality and rollback safety.
Why does GPU utilization look low during serving?	Decode is often memory-bound and request-driven; low SM utilization can still be optimal if latency is constrained. Check memory BW, kernel launch gaps, and batching policy.
When does quantization hurt?	If kernels fall back, if dtype boundaries add conversion overhead, or if quality drops force rework. I verify kernel coverage and quality gates before rollout.
TorchDynamo vs torch.export?	Dynamo captures runtime graphs from Python execution; export gives a cleaner, standardized graph for stable deployment transformations.
How do you debug scaling stalls after adding GPUs?	Measure compute/comm overlap, inspect NCCL traces, verify topology and message sizes, and adjust parallelism mode and micro-batch size.
What makes attention kernels fast?	Coalesced memory access, fused ops, low overhead KV-cache reads/writes, and good tensor core utilization for compatible shapes/dtypes.
How do you choose tensor vs pipeline parallelism?	Tensor parallel for large GEMMs and balanced layers; pipeline for very deep models or memory limits. Choice depends on interconnect and latency target.
How do you ensure safe deployment of optimizations?	Keep golden baselines, automated accuracy/perf gates, versioned artifacts, canary rollout, and fast rollback if SLOs or quality drift regress.
What would you optimize in TRT/TRT-LLM integration?	Graph lowering coverage, kernel selection heuristics, KV-cache policies, and reproducible benchmark harnesses across model families.
How do you reason about roofline for transformers?	Decode tends toward memory roof due to KV traffic; prefill can approach compute roof with larger batches and fused GEMM-heavy paths.
How do you improve user-perceived latency?	Continuous batching, token streaming, smart scheduling, and prioritizing p95 TTFT/token latency over only peak throughput.
What tests are mandatory for optimization code?	Numerical parity tolerances, shape/dtype coverage, regression benchmarks, and stress tests for long-context KV-cache behavior.

This rundown uses a practical lens for modern LLM and diffusion systems:

- Why raw GPU utilization can be misleading.
- Why large-parameter models choke on memory movement.
- Why data movement is frequently more expensive than arithmetic.

Core focus areas:

- **Memory Wall**
- **Distributed Inference**
- **Compiler and Kernel Optimization**

3 The Memory Wall and Locality

3.1 Why memory bandwidth is the first bottleneck

Key Idea

Core claim (preserved + clarified): The biggest hurdle for many inference kernels is memory bandwidth, not raw FLOPs.

Intuition:

- Compute units got much faster over time.
- Memory latency and bandwidth improved more slowly.
- A single load can cost hundreds of cycles, creating **memory stalls**.
- Most model weights cannot stay on-chip; they live in HBM/DRAM.

3.2 Latency vs bandwidth

- **Latency**: time to get one unit of data.
- **Bandwidth**: total volume moved per unit time.

Truck analogy (preserved): truck travel time is similar whether it carries few or many boxes. If your program requests tiny pieces of data repeatedly, you become latency-bound and inefficient.

4 Serving an LLM (Table Summary)

Key Idea

Big idea: Serving an LLM means turning a trained model into a fast, reliable API that real applications can call.

Stage	What Happens	Key Points / Interview Memory
Freeze Model	Save weights, tokenizer, configs; lock versions.	Start with FP16/BF16; use quantization later if memory/latency matters. Ensures reproducibility and stable deployment.
Inference Engine	Software that loads model onto GPUs and serves requests.	Example: **vLLM** — efficient batching, KV-cache optimization, high throughput. Provides API endpoint for apps.
Chain / App Layer	Business logic between app and model.	Build prompts, call tools (DB/search/code), format outputs, retries, moderation. Often FastAPI/Node.
RAG (Optional)	External knowledge retrieval before generation.	Chunk docs → embeddings → vector DB → retrieve relevant text → add to prompt. **Memory idea:** Search first, then generate.
Production Architecture	Typical serving pipeline.	App → API Gateway → Chain Service → LLM Server → GPU. Separates concerns for scalability.
Production Concerns	Operational considerations.	Latency: TTFT, tokens/sec. GPU usage: memory/utilization. Scaling: batching, multi-GPU. Safety: moderation, injection defense. Reliability: monitoring, retries.
Deployment Options	Infrastructure scale choices.	Single GPU (simple dev), multi-GPU node (higher throughput), distributed multi-node serving (large scale).

Key Idea

Interview one-liner:
"Serving an LLM means freezing the model, running it behind an optimized inference server, adding business logic or RAG in front, and monitoring latency, GPU usage, and reliability in production."

4.1 Memory hierarchy and locality

Level	Typical speed	Practical takeaway
Registers	Fastest, tiny capacity	Reuse values aggressively while they are hot
L1 / Shared memory	Very fast, small	Tile to keep working sets on-chip
L2 cache	Medium	Favor contiguous access to improve hit rate
HBM/DRAM	Slowest, largest	Expensive to touch repeatedly; reduce traffic

Tiered storage (quick table)

Tier	Used for	Tradeoff
Local NVMe	hot data + fast scratch I/O	fastest, limited capacity
Parallel FS	shared high-speed access	scalable, more complex
NFS	configs/scripts/small data	simple, not for heavy throughput
Object store	checkpoints/logs/big datasets	durable + cheap, higher latency

Important

Game (preserved): Keep data as high in the memory pyramid as possible.

- **Temporal locality**: reuse recently loaded data before eviction.
- **Spatial locality**: access nearby addresses so cache lines are fully used.
- **Stride pitfall**: jumping across memory hurts coalescing and wastes bandwidth.

5 GPU Execution Model: Why It Wins and Why It Stalls

5.1 CPU vs GPU model

- CPU: optimized for low-latency, branch-heavy, MIMD workloads.
- GPU: optimized for throughput, SIMD/SIMT-style parallelism.

Key Idea

Warp (preserved + clarified): A **warp** is 32 threads scheduled together. If branches diverge inside a warp, execution is serialized and throughput drops.

5.2 Occupancy and latency hiding

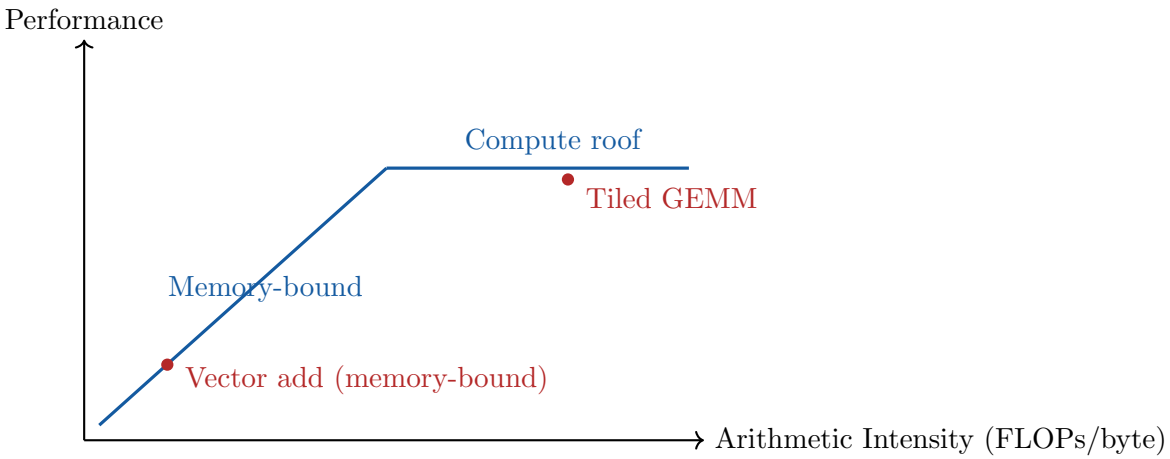
Occupancy is the number of active warps on an SM relative to hardware limits.

- If warp A waits on memory, the scheduler runs warp B (warp-dispatch latency hiding).
- You need enough active warps to hide memory latency.
- A kernel at 40% occupancy is not always bad, but it is a signal to inspect register/shared-memory pressure and memory behavior.

6 Roofline Model (Requested Graph Insert)

6.1 How to classify kernels

- X-axis: **Arithmetic Intensity** (FLOPs/byte).
- Y-axis: achieved performance (GFLOP/s or TFLOP/s).
- Slanted region: memory-bound.
- Flat roof: compute-bound peak.



Key Idea

Misconception cleared: More compute does not help if you are on the memory-bound slope. First reduce bytes moved or improve locality.

7 Precision and Quantization

7.1 Why lower precision matters

- Lower precision reduces model memory footprint.
- Smaller data types increase effective memory throughput.
- Tensor Cores typically use mixed-precision paths for speed + stability.

7.2 FP16 vs BF16

Important

FP32 fallback and low-precision caveats (preserved): Some operations are sensitive and often need higher precision for numerical stability:

- softmax
- normalization layers
- attention score calculations
- reductions

INT4 (two weights per byte) also introduces layout constraints:

- pack/unpack overhead
- alignment constraints
- avoid branch-heavy unpack logic that causes warp divergence

Format	Strength	Risk/Tradeoff
FP16	Smaller storage and high throughput	Narrow exponent range can underflow/overflow gradients
BF16	FP32-like exponent range with lower precision mantissa	Less precision than FP32, but usually better training stability than FP16

7.3 INT8/FP8 quantization

- **PTQ**: post-training quantization using calibration data.
- **QAT**: quantization-aware training to recover accuracy.

Important

Why quantization can fail or slow down (preserved + clarified):

- Kernel coverage gaps force dequantize/requantize overhead.
- If workload is compute-bound, memory savings may not move latency.
- Accuracy can drop due to outliers and poor calibration.

Best-case path: long contiguous subgraphs remain quantized end-to-end (compiler/runtime chooses this), minimizing format conversions.

8 Distributed Inference and Scaling Limits

Important

Very important insert (implemented): Data Parallelism

- Replicate the full model on each GPU; split inputs across replicas.
- Common for training and also useful for high-throughput inference serving.
- In training, gradients must be synchronized (typically with AllReduce-equivalent collectives).
- Communication cost depends strongly on interconnect: NVLink/NVSwitch, PCIe, Ethernet, or InfiniBand.

8.1 Model parallelism

Pipeline parallelism:

- Split layers across GPUs.
- Can suffer from pipeline bubbles if stages are imbalanced.
- Mitigated with micro-batching and careful schedule design.

Tensor parallelism:

- Split tensor dimensions of large matmuls across GPUs.
- Requires frequent communication of partial results.
- Needs high-bandwidth, low-latency links for efficiency.

Collective	**What it does**	**Common use**	**Easy recall / intuition**
Broadcast	One rank sends the same data to all other ranks	Weights, configs, initialization	"One → everyone" (copy data out)
AllReduce	Reduce across all ranks and store result in every rank	Gradient synchronization (data parallel training)	"Average grads everywhere" — each GPU trains mini-batch then syncs before update
Reduce	Reduce across ranks, result stored in one rank only	Aggregation, logging, metrics	"Everyone sends → one keeps"
AllGather	Gather shards so every rank ends with full data	Unsharding parameters/activations (FSDP, ZeRO-3)	"Pieces → full copy on every GPU"; often after sharded training step
ReduceScatter	Reduce first, then scatter shards so each rank keeps only a portion	Sharded gradient updates (FSDP, ZeRO)	"Reduce then split" — memory efficient; often followed by AllGather later

8.2 Strong scaling limit

Key Idea

Interview framing (preserved): At some point, adding GPUs makes each shard too small and communication dominates. Past that strong-scaling point, more GPUs can slow the job.

Interconnect cheat sheet

Tech	Connects	Why it matters
PCIe	CPU/host ↔ GPU	host-device bandwidth/latency bottleneck
NVLink	GPU ↔ GPU	high bandwidth for multi-GPU on-node
InfiniBand/RDMA	node ↔ node	faster distributed training communication
GPUDirect RDMA	GPU ↔ NIC	avoids host copies (lower latency, less CPU overhead)

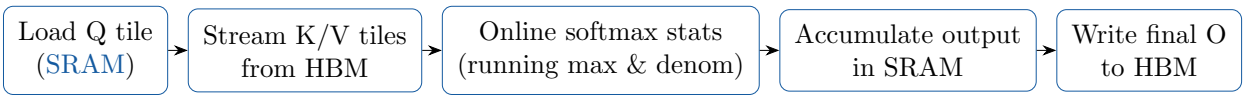
9 KV Cache and Decode Bottlenecks

Transformer attention (Q, K, V) in plain words

When the model reads each token, it creates Query (Q), Key (K), and Value (V) vectors. Keys and values help the model store what this token means. During attention, the current token's query is compared against other tokens' keys to decide what context matters. The model then uses the matched values to pass forward the information that matters. This is how the model tracks context and relationships across a sentence or conversation.

FlashAttention (tile-wise outline)

Process keys/values in tiles, maintain running softmax statistics, and accumulate the output in SRAM.



Key Idea

Why it works: keep tiles in fast memory (SRAM), do more compute per byte loaded, and avoid materializing the full attention matrix in HBM.

- During autoregressive decoding, new tokens attend to prior context.
- **KV cache** stores past key/value states to avoid recomputation.
- As context grows (e.g., 10k+ tokens), KV memory traffic becomes huge.

Important

Critical point: Decode is often memory-bound because each token requires reading large KV history for relatively limited compute.

Paged attention and cache-management strategies exist to improve locality and memory efficiency.

9.1 Prefill vs Decode

Phase	Primary bottleneck	Best optimization levers
Prefill	GEMM compute + activation memory	Better batching, tensor core utilization, graph capture, fusion
Decode	KV-cache bandwidth/latency	Paged KV cache, attention kernel fusion, reduced precision cache, continuous batching

10 Compiler Stack and Kernel Fusion

- Eager Python can launch many small kernels with overhead.
- `torch.compile` and graph capture reduce interpreter overhead.
- **Kernel fusion** combines ops (e.g., matmul + bias + activation) to reduce memory round-trips.

Key Idea

High-impact optimization (preserved): Fusion often shifts workloads right on the roofline plot by increasing arithmetic intensity and reducing memory traffic.

11 Sparsity Reality Check

11.1 Sparsity and Pruning

Type	Description	Practical impact
Unstructured sparsity	Removes individual weights	Often limited speedup unless specialized sparse kernels exist
Structured sparsity	Removes heads/channels/blocks	Better hardware mapping and real latency wins

- Unstructured sparsity can hurt GPU efficiency due to irregular memory access.
- GPUs prefer regular, coalesced access patterns.
- Structured patterns (for example, hardware-supported sparsity formats) are much easier to accelerate.

Interview signal: "Sparsity is only useful when compiler + kernel + hardware path is sparse-aware."

12 Numerical Stability and Reproducibility

- Floating-point addition is not associative in practice.
- Different reduction orders across devices can produce different results.
- Distributed collectives may be non-deterministic unless explicitly constrained.

Subnormal handling note (preserved + clarified): denormal/subnormal numbers can be very slow on some hardware paths. Flush-to-zero behavior can improve speed but may change tiny-value numerics; treat as a controlled tradeoff.

13 Final Mental Model

Important

Performance engineering is constant tradeoff management:

- compute vs memory traffic

Regime	What you see	What you do
Compute-bound	high SM active, BW not maxed, kernels dominate runtime	increase tensor core use, fuse kernels, improve occupancy, reduce divergence
Memory-bound	BW near peak, moderate SM active, lots of stalls	tiling, shared memory/cache reuse, coalescing, fewer global reads/writes, cachce misses
I/O-bound	GPU idle, data loader maxed, disk/network busy	dataset caching, preprocessing offline, parallel data loaders; faster storage (SSD/NVMe), reduced contention, and distributed data sharding with node-local caching

- communication vs parallelism
- numerical fidelity vs throughput
- Python-level abstraction vs hardware-level control

As Moore's law slows, gains increasingly come from software-hardware co-design and careful systems optimization.

Profiling Section

13.1 Profiling / Debugging Cheat Sheet

Tool / Snippet	Meaning (clean + interview-ready)
Nsight Systems (nsys)	End-to-end timeline: CPU↔GPU overlap, kernel launches, synchronization, memcpy, data loader stalls.
Nsight Compute (ncu)	Deep kernel analysis: occupancy, memory throughput, instruction mix, warp stalls, cache behavior.
cuda-gdb	Debug CUDA kernels (breakpoints, stepping, inspecting variables).
cuda-memcheck	Memory correctness: out-of-bounds, invalid accesses, (some) race-like issues.
CUDA runtime + driver APIs	Managing devices, memory, and program execution on the GPU.
cudaMalloc(...)	Allocates memory on the **GPU**. (`malloc` allocates on **CPU**.)
cudaMemcpy(...)	Copies data **CPU↔GPU** (host-device transfers).

Important

Fast rule: nsys tells you *where time goes overall.* ncu tells you *why a specific kernel is slow.*

13.2 Nsight Compute (NCU) Quick Commands

Key Idea

Use NCU to answer: am I compute-bound or memory-bound, and what limits occupancy/stalls?

What you want	Command
Profile app (basic)	`ncu ./vecAdd`
Fuller kernel metrics (heavier)	`ncu --set full ./vecAdd`
Profile only one kernel by name	`ncu -k vecAdd ./vecAdd`
Save report file	`ncu -o report_vecAdd ./vecAdd`
Reduce overhead (often faster)	`ncu --replay-mode application ./vecAdd`

13.3 CUDA Indexing (Fast Recall)

Name	Meaning
`gridDim.x`, `gridDim.y`	How many blocks wide/tall in the launched grid.
`blockDim.x`, `blockDim.y`	Threads per block in x/y.
`threadIdx.x`, `threadIdx.y`	Thread's local coordinates inside its block.
`blockIdx.x`, `blockIdx.y`	Block's coordinates inside the grid.

Key Idea

2D output indexing (matmul / conv-style):

$$i = \texttt{blockIdx.y} \cdot \texttt{blockDim.y} + \texttt{threadIdx.y}$$
$$j = \texttt{blockIdx.x} \cdot \texttt{blockDim.x} + \texttt{threadIdx.x}$$

13.4 Cluster Launch (Slurm) — `srun` Line Breakdown

Key Idea

Command (preserved):

```
srun --partition=gpu --gres=gpu:1 --mem=16G --time=01:00:00 --pty bash
```

Flag	What it does
`--partition=gpu`	Run on the GPU partition/queue.
`--gres=gpu:1`	Request 1 GPU resource.
`--mem=16G`	Request 16 GB of system RAM.
`--time=01:00:00`	Time limit of 1 hour.
`--pty bash`	Start an interactive pseudo-terminal (a shell) on the allocated node.

Important

Interview phrasing: "`srun --pty` is how I grab an interactive GPU node to debug quickly before running a long `sbatch` job."

insert CCUDA AMTMUL SECTION:

13.5 Naive Matrix Multiply (CPU + CUDA) — Interview Baseline

Key Idea

Purpose: show you understand indexing, memory layout, and where the bottleneck comes from. This is naive (no tiling/shared memory). Optimizations come after.

CPU Naive Matmul (row-major)

```
void matmul_cpu(const float* A, const float* B, float* C,
                int M, int K, int N) {
  // A: MxK, B: KxN, C: MxN (row-major)
  for (int i = 0; i < M; i++) {
    for (int j = 0; j < N; j++) {
      float sum = 0.0f;
      for (int k = 0; k < K; k++) {
        sum += A[i*K + k] * B[k*N + j];
      }
      C[i*N + j] = sum;
    }
  }
}
```

CUDA Naive Matmul Kernel

```
__global__ void matmul_cuda(const float* A, const float* B, float* C,
                            int M, int K, int N) {
  int i = blockIdx.y * blockDim.y + threadIdx.y; // row
  int j = blockIdx.x * blockDim.x + threadIdx.x; // col

  if (i < M && j < N) {
    float sum = 0.0f;
    for (int k = 0; k < K; k++) {
      sum += A[i*K + k] * B[k*N + j];
    }
    C[i*N + j] = sum;
  }
}
```

Launch Configuration (simple)

```
// Example: 16x16 threads per block
dim3 block(16, 16);
dim3 grid((N + block.x - 1) / block.x,
          (M + block.y - 1) / block.y);

matmul_cuda<<<grid, block>>>(A, B, C, M, K, N);
```

Important

What makes this naive kernel slow:

- Each thread re-loads A/B from global memory many times (low data reuse).
- No shared-memory tiling, so memory traffic is high.
- Typically becomes memory-bound at scale (unless extremely small sizes).

13.6 Simple Tiling + Shared Memory Optimization (next step)

Key Idea

Goal: reuse global memory loads by staging tiles of A and B in shared memory.
Each block loads a small tile once, then reuses it for many MACs.

Tiled CUDA Matmul (shared memory)

```
#define TILE 16

__global__ void matmul_tiled(const float* A, const float* B, float* C,
                             int M, int K, int N) {
  __shared__ float As[TILE][TILE];
  __shared__ float Bs[TILE][TILE];

  int row = blockIdx.y * TILE + threadIdx.y;
  int col = blockIdx.x * TILE + threadIdx.x;

  float sum = 0.0f;

  // Loop over tiles in the K dimension
  for (int t = 0; t < (K + TILE - 1) / TILE; t++) {

    int a_col = t * TILE + threadIdx.x; // k index for A
    int b_row = t * TILE + threadIdx.y; // k index for B

    // Load A and B tiles into shared memory (with bounds checks)
    As[threadIdx.y][threadIdx.x] =
        (row < M && a_col < K) ? A[row * K + a_col] : 0.0f;

    Bs[threadIdx.y][threadIdx.x] =
        (b_row < K && col < N) ? B[b_row * N + col] : 0.0f;

    __syncthreads(); // ensure tiles are loaded before compute

    // Compute partial dot product for this tile
```

```
    #pragma unroll
    for (int k = 0; k < TILE; k++) {
      sum += As[threadIdx.y][k] * Bs[k][threadIdx.x];
    }

    __syncthreads(); // ensure all threads finished before loading next tile
  }

  if (row < M && col < N) {
    C[row * N + col] = sum;
  }
}
```

Launch Configuration (tiled)

```
dim3 block(TILE, TILE);
dim3 grid((N + TILE - 1) / TILE,
          (M + TILE - 1) / TILE);

matmul_tiled<<<grid, block>>>(A, B, C, M, K, N);
```

Important

Why shared-memory tiling is faster (simple recall):

- Global → shared once per tile, then reused many times.
- Much higher arithmetic intensity (more FLOPs per byte loaded).
- Less pressure on HBM/global bandwidth → closer to compute-bound.

Experience Section

HPC Workflows: Multi-GPU Training + Benchmarking (WAVE HPC Support Experience)

Slurm, Kubernetes, GPU clusters, distributed ML infrastructure

13.7 Short Pitch

I worked on designing and supporting HPC workflows for multi-GPU training, benchmarking, and cluster resource optimization while also serving as a support engineer on the WAVE HPC cluster. The role involved helping users run distributed ML workloads, troubleshooting job scheduling issues, and improving GPU utilization through better profiling, scheduling practices, and workflow design.

13.8 Why I Did It + What I Learned

I wanted hands-on exposure to real GPU cluster infrastructure beyond just running experiments myself, and working in a support role at WAVE gave me visibility into how many different workloads

behave at scale. It helped me understand how scheduling policies, communication overhead, data pipelines, and storage systems affect distributed deep learning performance. Supporting other users also strengthened my ability to explain HPC concepts in a beginner-friendly way while still thinking from a systems and performance perspective.

13.9 Challenges

One challenge was diagnosing distributed training slowdowns, which often came from communication overhead, data loading bottlenecks, or misconfigured jobs rather than compute limits. Load balancing across GPUs and nodes also required careful monitoring. Another challenge was ensuring reproducibility across environments, since differences in drivers, libraries, and cluster configurations could affect results.

13.10 Key Interview Takeaways

A key idea I emphasize is that scaling ML workloads introduces communication and infrastructure bottlenecks that can offset raw compute gains. Efficient cluster usage often requires profiling, scheduling optimization, and good workflow design, not just bigger models or faster GPUs. My WAVE support experience reinforced how real-world HPC systems operate and how to think about performance from both the user and infrastructure perspective.

Research: LLMCompass — Computational Graph Exploration Across LLMs and Hardware

LLM systems, hardware–model co-design, inference optimization, GPU/HPC performance

13.11 Short Pitch (GPU / HPC Focus)

I worked on LLMCompass, a research-oriented computational graph tool designed to explore how different large language models map onto different hardware systems. The goal was to build intuition around how model architecture, inference strategies, and hardware characteristics interact, especially in terms of GPU utilization, memory bandwidth, and system throughput.

13.12 Why I Did It + What I Learned

I wanted a beginner-friendly but research-grounded way to understand how LLM inference actually behaves on real hardware beyond just reading papers. Working on LLMCompass helped me see how computational graphs translate into memory movement, compute patterns, and scaling behavior. I gained intuition about how model structure, batching, and hardware characteristics influence overall performance.

13.13 Challenges

One challenge was understanding how different hardware constraints change the optimal computational graph design. Another was learning to interpret performance tradeoffs without assuming one model or hardware setup is universally best. I also worked on keeping explanations accessible so newer students could use the tool to build systems intuition.

13.14 Key Interview Takeaways

A key idea I emphasize is that LLM performance is heavily shaped by the interaction between model architecture and hardware capabilities, not just model size alone. Computational graphs help visualize these tradeoffs, making it easier to reason about bottlenecks like memory bandwidth, compute utilization, and scaling across accelerators.

Research: Convolution Optimization with Triton

Triton, CUDA, PyTorch, GPU kernel optimization, memory hierarchy

13.15 Short Pitch

I worked on implementing custom Triton kernels for 2D convolution layers to better understand how deep learning workloads map onto GPUs. The project focused on exploring memory hierarchy, tiling strategies, and thread mapping to improve GPU utilization and reduce kernel overhead compared to standard PyTorch convolution operations.

13.16 Why I Did It + What I Learned

I wanted hands-on experience writing GPU kernels in a way that was beginner-friendly but still grounded in real performance research. Since most frameworks rely on cuDNN as a black box, this helped me build intuition about how convolutions actually execute on hardware. I learned how memory access patterns, tiling, and warp-level execution influence performance, and developed a stronger understanding of GPU memory locality, tensor core usage, and kernel scheduling tradeoffs.

13.17 Challenges

One challenge was balancing occupancy with shared memory and register limits while still maintaining good performance. Debugging correctness while optimizing speed was also tricky, especially when working with custom kernel implementations. Another challenge was understanding when the kernel was compute-bound versus memory-bound and how that affected optimization decisions.

13.18 Key Interview Takeaways

A key idea I emphasize is that GPU kernel optimization is largely about memory locality and efficient data movement rather than just raw compute. Writing Triton kernels made it much clearer how tiling, coalesced memory access, and batching influence throughput, and why profiling tools like Nsight Compute are essential for understanding performance bottlenecks.

GPU-Accelerated Image Processing Pipeline

CUDA, PyTorch, GPU kernels, memory optimization, V100 GPUs

13.19 Short Pitch

I built a GPU-accelerated image processing pipeline implementing blur and sharpen filters using custom CUDA kernels, achieving up to an 8× speedup over CPU baselines. The project focused on understanding how image processing workloads map onto GPUs and how memory access patterns influence performance.

13.20 Why I Did It + What I Learned

I wanted strong GPU fundamentals before going deeper into ML systems work, so this project was intentionally beginner-friendly but grounded in real performance engineering. It helped me understand kernel launch configuration, GPU memory hierarchy, profiling workflows, and when GPU acceleration is actually worthwhile. I also developed intuition about data locality, host-device transfers, and how preprocessing pipelines interact with GPU compute.

13.21 Challenges

One challenge was optimizing memory access patterns so threads could efficiently process image pixels without excessive global memory traffic. Avoiding unnecessary CPU–GPU transfers was another important factor for performance. I also focused on stable benchmarking across different image sizes while validating GPU outputs against CPU reference implementations to ensure correctness.

13.22 Key Interview Takeaways

A key idea I emphasize is that GPU performance is often limited more by memory movement than raw compute, especially in image processing pipelines. Writing custom CUDA kernels made it clearer how thread mapping, shared memory caching, and memory coalescing directly affect throughput and why profiling is essential when optimizing GPU workloads.

High-Performance Matrix Inversion via Custom GPU Kernels

Triton, OpenCL, C++, GPU linear algebra, HPC optimization

13.23 Short Pitch

I worked on implementing GPU-based matrix inversion kernels for large matrices, focusing on how numerical linear algebra workloads map onto GPU hardware. The project explored parallel decomposition strategies, memory reuse, and kernel design to reduce runtime compared to traditional CPU-based numerical implementations while building intuition around HPC-style compute patterns relevant to machine learning systems.

13.24 Why I Did It + What I Learned

I wanted a beginner-friendly but research-grounded way to understand how foundational linear algebra operations — like matrix inversion and multiplication — connect directly to deep learning performance. This project helped me see how many ML workloads ultimately reduce to optimized matrix operations, and how GPU kernel design, tiling, and memory hierarchy decisions impact both training and inference efficiency. I also gained perspective on portability versus performance by experimenting with both Triton and OpenCL approaches.

13.25 Technical Implementation (Code Logic)

The implementation partitions matrices into blocks processed in parallel by GPU threads, where each thread block performs partial inversion or elimination steps. Shared memory is used to cache intermediate values and reduce global memory traffic, while synchronization ensures correctness across blocks. I also explored kernel fusion techniques to minimize intermediate writes, and compared

OpenCL implementations to understand tradeoffs between cross-platform portability and peak GPU performance.

13.26 Challenges

Maintaining numerical stability while pushing for performance was one of the main challenges, especially when balancing precision and throughput. Synchronization overhead, memory reuse patterns, and choosing appropriate block sizes also required careful tuning. Comparing different frameworks introduced additional complexity around portability and optimization tradeoffs.

13.27 Key Interview Takeaways

A key idea I emphasize is that many large-scale ML workloads are fundamentally linear algebra problems, so optimizing these primitives directly impacts system performance. This project reinforced how kernel tiling, synchronization strategy, memory locality, and precision management all play major roles in GPU efficiency and scalability.

LABUBU AI Shopper

LLMs, vector search, FastAPI, React, Faiss, RAG systems

13.28 Short Pitch

I built LABUBU AI Shopper, a real-time AI shopping assistant that combines vector search with LLM prompting to help users find products conversationally. The system handled around 1000 SKUs with sub-second response times, focusing on making LLM-powered retrieval practical for a production-style application.

13.29 Why I Did It + What I Learned

I wanted hands-on experience building a full LLM application stack in a way that was beginner-friendly but still reflective of real production systems. Through this project, I learned prompt engineering, retrieval-augmented generation design, and latency optimization across both the vector search and inference pipeline. It also gave me a better intuition for the tradeoffs between embedding search cost, caching strategies, and overall response speed.

13.30 Challenges

One challenge was managing latency across multiple stages — retrieval, reranking, and LLM inference — while keeping responses fast enough for real-time interaction. Scaling the vector search while maintaining result quality was another consideration. I also focused on designing a reliable API workflow so the frontend and backend stayed responsive under load.

13.31 Key Interview Takeaways

A key idea I emphasize is that in many real LLM systems, retrieval latency can dominate inference latency, so optimizing data flow and caching is just as important as optimizing the model itself. Building this system helped me understand how RAG pipelines work end-to-end and how architectural decisions affect both performance and user experience.